

Aplicaciones con comunicaciones seguras



Caso práctico

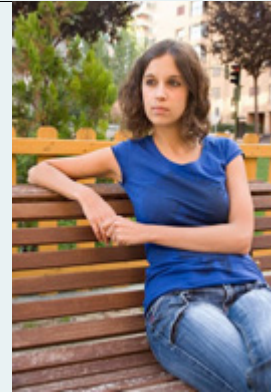
Durante todo el fin de semana, **Ana** ha estado dándole vueltas a la nueva tarea que **Ada** le ha propuesto que realice junto a **Juan**. Se trata de modificar algunas partes de una antigua aplicación cliente/servidor de una Gestoría de Seguros, que utiliza el personal interno en su trabajo diario.

Aunque la aplicación en su momento, según comentan, era bastante segura y utilizaba bibliotecas criptográficas para garantizar la confidencialidad e integridad de los datos transmitidos por la Intranet, en la actualidad han podido comprobar que se ha vuelto vulnerable y puede ser objeto de ataques por parte de intrusos sin dificultad.

Para **Ana** aquello más llamativo ha sido tener conocimiento de que el hijo adolescente de un empleado, entusiasta de la informática y las tecnologías de la información, fue el que avisó de dicho problema a su madre. Esperando dentro de la oficina a su madre fue capaz de interceptar, en menos de cuatro horas, el tráfico de la red interna. Ha logrado descifrar contraseñas e introducirse en los archivos de datos del servidor principal. Y todo esto lo ha podido hacer simplemente utilizando software libre disponible en Internet, al alcance de todos.

Ana suspira —¡claro!, al ser la aplicación antigua, no utiliza las últimas bibliotecas criptográficas y por tanto con la velocidad de cómputo actual, lo que hace unos años era seguro, hoy en día puede correr peligro. Además, puede que también tenga algún que otro fallo de seguridad.

Ana lo tiene claro, debe comenzar a repasar los objetivos de seguridad que debe cumplir una aplicación para que sus comunicaciones sean seguras. Sin más remedio tendrá que volver a repasar criptografía.



Q Ministerio de Educación (Uso educativo nc)

**Materiales formativos de FP Online propiedad del Ministerio de Educación,
Formación Profesional y Deportes.**

 [Aviso Legal](#) 

1.- Introducción

Cuando desarrollamos aplicaciones con comunicaciones, por ejemplo a través de Internet, debemos proporcionar seguridad tanto a la aplicación como a los datos transmitidos, ya que las operaciones que se realizan a través de la red podrían ser interceptadas, y por tanto manipuladas por personas desautorizadas.

La **seguridad de una aplicación y de los datos transmitidos** dependerá de su diseño, la selección de protocolos de comunicación y el método de autenticar al usuario.



Q Sebastián Piraña (
CC BY-NC-SA)

Los **objetivos de seguridad** que debe proporcionar una aplicación con comunicaciones seguras son los siguientes:

- **Confidencialidad.** Consiste en garantizar que los datos transmitidos por la aplicación sólo van a estar disponibles para las entidades (personas o procesos) autorizadas a acceder a dicha información. Si esos datos son interceptados por entidades desautorizadas, éstas no podrán acceder a la interpretación de esa información.
- **Integridad.** Consiste en garantizar que los datos originales no han sido modificados o alterados por alguna entidad no autorizada durante su transmisión.
- **Autenticación.** Consiste en asegurar que la entidad emisora es quien dice ser.
- **No repudio.** Consiste en asegurar que las acciones de un usuario han sido realizadas exactamente por dicho usuario. El no repudio garantiza la participación de las partes en una comunicación. En toda comunicación existe un emisor y un receptor, por lo que se pueden distinguir dos tipos de repudio:
 - **No repudio en origen:** garantiza que la persona que envía el mensaje no puede negar que es el emisor del mismo, ya que el receptor tendrá pruebas del envío.
 - **No repudio en destino:** el receptor no puede negar que recibió el mensaje, porque el emisor tiene pruebas de la recepción del mismo.
- **Autorización.** Trata de asegurar que una entidad puede realizar una acción en concreto, esto es, la autorización valida las acciones del usuario para verificar si tiene privilegios para realizar dicha acción.

Veamos los 4 primeros objetivos con un **ejemplo sencillo**:

- Cuando Antonio le envía información a Isabel, debe asegurarse de que esa información no es alterada o manipulada por el camino (Integridad de la información transmitida).
- La información es para Isabel, por tanto nadie más debe entender el mensaje (Confidencialidad).
- Debe haber alguna indicación de que el mensaje proviene de Antonio, esto es, debe haber alguna prueba de ello (Autenticación). La autenticación de

que el mensaje proviene de Antonio es proporcionado por lo que se denomina firma digital.

- ¿Quién garantiza la identidad y la firma de Antonio? Esto se logra mediante un certificado digital, que autentica la firma de Antonio.
- Además, es igualmente importante garantizar que Antonio no pueda negar la autoría del envío del mensaje, indicando que alguien lo envió en su nombre. (No repudio). Este objetivo también se logra con la firma digital.

En esta unidad veremos la forma de conseguir estos objetivos de seguridad utilizando **criptografía**, uno de los pilares básicos sobre los que descansan la mayoría de las soluciones de seguridad.

Y, ¿qué es o en qué consiste la criptografía? Esto es lo que vamos a comenzar a ver en el siguiente apartado.



Autoevaluación

La confidencialidad garantiza que la información transmitida no ha sido alterada por el camino.

☐ Verdadero ☐ Falso

Falso

Es falso, de lo que trata la confidencialidad es de garantizar que la información transmitida solo podrá ser interpretada por el destinatario.

2.- Criptografía



Caso práctico



Q Ministerio de
Educación (Uso educativo nc)

interesado en aprenderlo. ¡Esta vez, vas a ser tú, Ana, la que hagas un poco de maestra!

Nada más llegar a la empresa, **Ana** se ha reunido con **Juan** para planificar el nuevo trabajo que el viernes les encomendó **Ada**. Juan ha escuchado atentamente a **Ana** y asintiendo con la cabeza le ha dicho —Me parece perfecto comenzar repasando las posibilidades y diferentes aplicaciones de la criptografía. Cuando estudié le ciclo de DAI antiguo, no vimos este tema o muy de pasada y realmente siempre he estado

¿Qué es la criptografía? El término **criptografía** viene del griego "cripto" (secreto) y "grafía" (escritura), por lo que su significado es "escritura secreta". La finalidad de la criptografía es enmascarar o codificar una información original, utilizando alguna técnica, de manera que el resultado sea ininteligible para las personas no autorizadas.



Q zaphad1 (CC BY-NC-SA)

La criptografía se ha usado a lo largo de los años para mandar mensajes confidenciales o privados. Básicamente, cuando alguien quiere mandar información confidencial de forma secreta, actúa de la siguiente manera:

- Aplica técnicas criptográficas para poder "enmascarar" el mensaje.
- Manda el mensaje "enmascarado" por una línea de comunicación que se supone insegura y puede ser interceptada.
- Solo el receptor autorizado podrá leer el mensaje "enmascarado".

Otra disciplina relacionada con la criptografía es el **criptoanálisis**, que analiza la robustez de los sistemas criptográficos y comprueba si realmente son seguros. Para ello, se intenta romper la seguridad que proporciona la criptografía, deshaciendo el sistema y accediendo de esta forma a la información secreta en su formato original. Esta disciplina es una herramienta muy poderosa que permite mejorar los sistemas de criptografía constantemente y ayuda a desarrollar otros nuevos más efectivos.

Por último, y para terminar de recorrer estas disciplinas, debes conocer el término **criptología**, que es la ciencia que engloba tanto las técnicas de criptografía como las de criptoanálisis.



Autoevaluación



La criptografía consiste en enmascarar la información mediante alguna técnica, de manera que solo el destinatario autorizado pueda leer la información original.

☐ Verdadero ☐ Falso

Verdadero

Es verdadero, precisamente la criptografía se ha usado a lo largo de los años para mandar mensajes de forma secreta.



Para saber más

El siguiente vídeo, desarrollado por [intypedia](#), hace un recorrido por la historia de la criptografía y su desarrollo en Europa.

<https://www.youtube.com/embed/a99Qorfotv4>
(<https://www.youtube.com/embed/a99Qorfotv4>)

Q Universidad Politécnica de Madrid (UPM)

(Licencia estándar de YouTube)

 Descripción textual del vídeo 

2.1.- Encriptación de la información

La **encriptación o cifrado de la información** es el proceso por el cual la información o los datos a proteger son traducidos o codificados como algo que parece aleatorio y que no tiene ningún significado (datos encriptados).

La **desencriptación o descifrado** es el proceso inverso, en el cual los datos encriptados son convertidos nuevamente a su forma original.

A continuación, te indicamos los conceptos o términos asociados con la encriptación:

- **Texto llano o claro:** es la información original, la que no está cifrada o encriptada.
- **Criptograma o texto cifrado:** es la información obtenida tras la encriptación.
- **Algoritmo criptográfico o algoritmo de cifrado:** es un conjunto de pasos u operaciones, normalmente una función matemática, usado en los procesos de encriptación y desencriptación. Asociado al algoritmo hay una **clave o llave** (un número, palabra, frase, o contraseña).
- La **clave** controla las operaciones del algoritmo dentro del proceso de cifrado y descifrado, de manera que usando el mismo algoritmo con llaves diferentes, se obtienen textos cifrados o criptogramas diferentes.
- La clave puede ser simétrica (misma llave para cifrar y descifrar) o asimétrica (llaves diferentes para cifrar y descifrar).

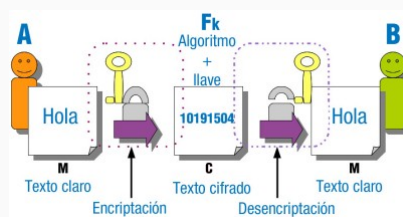
Resumiendo, podemos representar este proceso de la siguiente forma:

$$\text{Cifrado} \rightarrow F_k(M) = C$$

Q Ministerio de Educación (Uso educativo nc)

Lo que indica que el cifrado consiste en aplicar un algoritmo F , controlado por una llave K , sobre un mensaje o texto claro M , para obtener un criptograma o texto cifrado C . El descifrado, será el proceso inverso.

El siguiente esquema, muestra un ejemplo muy sencillo de encriptación con clave simétrica, donde **el algoritmo se representa mediante un candado y la clave mediante una llave**.



Q Ministerio de Educación (Uso educativo nc)

En este ejemplo podrían ser:

- Algoritmo: posición de la letra en el alfabeto representada con dos dígitos.
- Clave: 3.
- Algoritmo Cifrado: posición de la letra en alfabeto (dos dígitos) + 3.
- Descifrado: posición de la letra en alfabeto (dos dígitos) - 3.

Así tendríamos que:

H	o	l	a
$7 + 3 \rightarrow 1$	$16 + 3 \rightarrow 19$	$12 + 3 \rightarrow 15$	$1 + 3 \rightarrow 04$

En el siguiente enlace puedes ver otro ejemplo de todo esto:

[Ejemplo sencillo de cifrado de un mensaje](#)



Citas para pensar

"Cuando leemos demasiado deprisa o demasiado despacio, no entendemos nada".

Blaise Pascal



Autoevaluación

Un algoritmo criptográfico realmente es una clave o llave.

☐ Verdadero ☐ Falso

Falso

Es falso, la llave va asociada al algoritmo y controla las operaciones del proceso de cifrado y descifrado.

2.2.- Principios criptográficos

Las propiedades deseables de un sistema criptográfico las podemos resumir en los siguientes principios, que fueron enunciados por Auguste Kerckhoffs en 1883 y que dicen:

- Si el sistema no es teóricamente irrompible, al menos debe serlo en la práctica.
- La efectividad del sistema no debe depender de que su diseño permanezca en secreto.
- La clave debe ser fácilmente memorizable de manera que no haya que recurrir a notas escritas.
- Los criptogramas deberán dar resultados alfanuméricos.
- El sistema debe ser operable por una única persona.
- El sistema debe ser fácil de utilizar.



Q Ian Kennedy (
CC BY-NC-SA)

El segundo principio recibe el nombre de **principio de Kerckhoffs** y establece que **la seguridad o fortaleza del sistema criptográfico debe depender exclusivamente de mantener en secreto la clave y no de ocultar el diseño del sistema**, que puede ser público y conocido. De hecho, en la actualidad, la mayoría de los algoritmos utilizados en criptografía son de dominio público.

Ese principio fue reformulado posteriormente por Claude Shannon de la siguiente forma:

- **El adversario o enemigo conoce el sistema.**

A esta última formulación se la conoce como la máxima de **Shannon** y es el principio más adoptado por los criptólogos en oposición a la llamada seguridad a través de la oscuridad, basada en la ocultación del diseño e implementación del sistema criptográfico a los usuarios.

Entonces, ¿de qué depende realmente la **seguridad de un sistema criptográfico**? Depende de dos factores:

- **El diseño o robustez del algoritmo.** Cuanto menos fallos tenga el algoritmo, más seguro será el sistema.
- **La longitud de la clave utilizada.** A mayor longitud de la clave, mayor seguridad proporcionará el sistema. Actualmente, esta longitud se estima que debe ser como mínimo de 128 bits, existiendo algoritmos que permiten seleccionar el tamaño de la clave.

En la actualidad, se utilizan fundamentalmente dos métodos criptográficos: el conocido como **cifrado simétrico o de clave privada** y el conocido como **cifrado**

asimétrico o de clave pública. Veremos estos métodos en los siguientes apartados.



Debes conocer

En el siguiente enlace encontrarás una explicación de lo importante que es la longitud de una clave en un sistema de cifrado. También se explica lo que se conoce como 'ataque de fuerza bruta':

Importancia de la longitud de la clave en criptografía (http://es.wikipedia.org/wiki/Longitud_de_la_clave).



Autoevaluación

Uno de los principios criptográficos es:

- ☐ El algoritmo de encriptación debe ser complejo y no conocido.
- ☐ La clave es conocida.
- ☐ La seguridad del sistema depende de la complejidad del algoritmo.
- ☐ El diseño del sistema puede ser público.

No es correcto, prueba de nuevo.

No es correcto, deberías haber leído mejor.

No es correcto, debes poner más atención.

Correcto. Efectivamente, el diseño puede ser público, ya que la efectividad del sistema debe depender de mantener en secreto la clave.

Solución

1. Incorrecto (#answer-15_63)
2. Incorrecto (#answer-15_83)

3. Incorrecto (#answer-15_86)

4. Opción correcta (#answer-15_89)

2.3.- Criptografía de clave privada o simétrica

Este método de encriptación utiliza una clave secreta, conocida solo por emisor y receptor, para encriptar la información. Se la denomina también criptografía simétrica



Q Ministerio de Educación (Uso educativo nc)

porque **la clave utilizada para la encriptación y descifrado es la misma**. Es adecuada para garantizar confidencialidad.

A continuación te indicamos los principales aspectos de la **criptografía de clave privada o simétrica**:

■ Características:

- La clave es privada o secreta, solo la conocen las partes involucradas.
- Se utiliza la misma clave para el cifrado y descifrado. Así, según el gráfico de arriba, cuando A le envía información a B, utiliza la clave privada para cifrar el mensaje, que solo podrá descifrar B si también conoce la clave. Por tanto, A debe hacer llegar a B la clave privada.

■ Ventaja:

- Son algoritmos muy rápidos y que no aumentan el tamaño del mensaje; por tanto adecuados para cifrar grandes volúmenes de datos.

■ Inconvenientes:

- El problema de la distribución de claves: el receptor debe conocer la clave que se va a utilizar, lo que implica que el emisor se la debe enviar y no es posible utilizar medios inseguros para el intercambio de claves.
- Otro inconveniente es el gran número de claves que se necesitaría si el grupo de personas que puede comunicarse de forma privada es muy grande, pues se necesitaría una clave diferente cada dos personas del grupo.

- Algunos **ejemplos de algoritmos** de este tipo son: AES o **Rijndael** (Nuevo estándar mundial), DES, 3-DES, DES-X, IDEA y RC5.



Autoevaluación

La criptografía de clave privada utiliza dos claves diferentes para cifrar y descifrar la información.

☐ Verdadero ☐ Falso

Falso

Es falso, la clave que se utiliza es la misma.



Para saber más

El siguiente vídeo, desarrollado por [intypedia](#), explica de forma detallada los fundamentos de la criptografía de clave privada o simétrica:

<https://www.youtube.com/embed/46Pwz2V-t8Q>
(<https://www.youtube.com/embed/46Pwz2V-t8Q>).

Q Universidad Politécnica de Madrid (UPM)

(Licencia estándar de YouTube)

 Descripción textual del vídeo 

2.4.- Criptografía de clave pública o asimétrica

La criptografía de clave pública surge para solucionar el problema de distribución de claves que plantea la criptografía de clave privada, permitiendo que emisor y receptor



Q Ministerio de Educación (Uso educativo nc)

puedan acordar una clave en común sobre canales inseguros. Se la denomina también criptografía asimétrica porque **las claves utilizadas para la encriptación y descryptación son diferentes**. Es adecuada para garantizar además de confidencialidad, la autenticación de los mensajes y el no repudio.

A continuación, te indicamos los aspectos más destacados de la **criptografía de clave pública o asimétrica**.

■ Características:

- Cada parte posee una pareja de claves, una pública, conocida por todos, y su inversa privada, conocida solo por su poseedor.
- Cada pareja de claves, son complementarias: lo que cifra una de ellas, solo puede ser descifrado por su inversa.
- Esa pareja de claves sólo se puede generar una vez, de modo que se puede asumir que no es posible que dos personas hayan obtenido casualmente la misma pareja de claves.
- Al cifrar un mensaje con la clave pública, tan solo podrá descifrarlo quien posea la clave privada inversa a esa clave pública. Así, según el gráfico de arriba, cuando A le envía información a B, utiliza la clave pública de B para cifrar el mensaje, que solo podrá descifrar B con su clave privada.
- Conocer la clave pública no permite obtener ninguna información sobre la clave privada, ni descifrar el texto que con ella se ha cifrado.
- Cifrar un mensaje con la clave privada equivale a demostrar la autoría del mensaje, autenticación, nadie más ha podido cifrarlo utilizando esa clave privada. El mensaje cifrado con una clave privada podrá descifrarlo todo aquel que conozca la clave pública inversa. Por tanto, si A enviase un mensaje cifrado con su clave privada, podrá descifrarlo todo el que conozca la clave pública de A y además sabrá que el mensaje proviene de A.

■ Ventaja:

- No existe el problema de distribución de claves, ya que cada parte posee su juego de claves.

■ Inconvenientes:

- Son más lentos que los algoritmos simétricos.
- Garantizar que la clave pública es realmente de quien dice ser su poseedor, lo que se conoce como el problema del 'ataque del hombre en el medio' o man in the middle. (Es un ataque en el que el atacante es capaz de observar e interceptar mensajes entre las dos partes sin que ninguna de ellas sepa que el enlace entre ellos ha sido violado).

- Algunos ejemplos de algoritmos de este tipo son: DSA, RSA(estándar de facto), algoritmo de Diffie-Hellman.

Generalmente se utiliza una combinación de ambas: criptografía asimétrica para negociar una clave privada con la que después se comunicarán los datos.



Autoevaluación

La criptografía de clave pública utiliza claves diferentes para cifrar y descifrar la información.

☐ Verdadero ☐ Falso

Verdadero

Es verdadero, una clave es pública y la otra es su inversa privada.



Para saber más

El siguiente vídeo, desarrollado por [intypedia](#), explica de forma detallada los fundamentos de la criptografía de clave pública o asimétrica:

<https://www.youtube.com/embed/On1clzor4x4>
(<https://www.youtube.com/embed/On1clzor4x4>)

Q Universidad Politécnica de Madrid (UPM)

(Licencia estándar de YouTube)

 Descripción textual del vídeo 

2.5.- Resumen de mensajes, firma digital y certificados digitales

Otras técnicas criptográficas son las siguientes:

■ Resumen de mensajes, basado en funciones

HASH. Un algoritmo de resumen toma como entrada un mensaje de longitud variable y lo convierte en un resumen de longitud fija, cuyas principales características son: siempre debe proporcionar la misma salida, debe ser aleatorio y unidireccional.



Q Mario A. P. (CC BY-NC-SA)

- El principal uso que tienen es proporcionar integridad.
- Combinado con técnicas de clave pública se obtiene una forma eficiente de proporcionar identificación.
- Los algoritmos HASH o de resumen más usados son MD5 y SHA.

■ Firmas digitales.

Son el equivalente digital de las firmas personales. Se basan en criptografía de clave pública y resumen de mensajes o funciones HASH.

- Para la transmisión de un mensaje entre un emisor y un receptor, el emisor transmitirá, junto con el texto deseado, la firma digital del mensaje cuya finalidad es comprobar la integridad del mensaje y la autenticidad del emisor.
- Para ello, el emisor debe disponer de una clave pública y otra privada. Cuando desee enviar un mensaje a un receptor, codificará el mensaje con una función HASH cuya salida la cifrará con su clave privada, generando así la firma digital que transmitirá al receptor junto con el texto deseado. El receptor, separará el mensaje recibido en dos partes: el texto y la firma. Usará la clave pública del emisor para descifrar la firma y, al texto que recibe le aplicará la misma función HASH que el emisor, comparando la salida de su función con el mensaje descifrado incluido en la firma. Si coinciden, quedará probada la integridad del mensaje y la autenticidad del emisor.

■ Certificados digitales.

Pretenden resolver el problema de la confianza entre las partes, delegando la responsabilidad en un tercero. Es decir, un certificado digital no es más que un mensaje firmado por una parte de una conversación, especie de notario digital que autentifica a las partes, que certifica que una clave pública pertenece a quién dice ser su poseedor. Para ello, los certificados digitales, según el estándar **X.509**, deben contener entre otra la siguiente información: número de versión, número de serie del certificado, información del algoritmo del emisor, emisor del certificado, periodo de validez, información sobre el algoritmo de clave pública, firma digital de la autoridad emisora.

Con todo esto, se introduce el nuevo concepto de **Entidad Certificadora**, que son organizaciones que se responsabilizan de la validez de los certificados, teniendo que, además de crearlos, proporcionar un mecanismo que permita su revocación, su suspensión, la búsqueda de certificados y la comprobación del estado del certificado.



Debes conocer

Desde el siguiente enlace puedes ver la diferencia entre encriptar y firmar la información:

 [Diferencia entre encriptar y firmar la información](http://quizasnosabiasque.blogspot.com/2010/07/conoces-la-diferencia-entre-firmar-y.html)  (<http://quizasnosabiasque.blogspot.com/2010/07/conoces-la-diferencia-entre-firmar-y.html>).



Para saber más

En este documento, desarrollado con información extraída de la sección 'Preguntas Frecuentes' de la 'Fábrica Nacional de Moneda y Timbre' y publicado por el 'Servicio Central de Informática' de la Universidad de Málaga, puedes ver ejemplos muy ilustrativos del uso de certificados y firmas digitales así como de otros aspectos estudiados anteriormente:

 [Documento con información sobre firma y certificado digital](#) 
([PSP07_CONT_R012_Firma_Certificado_Digital.pdf](#)) (0.37 MB)

En este otro enlace obtendrás información sobre la tecnología PKI en el sitio web Wikipedia:

[Información sobre PKI](http://es.wikipedia.org/wiki/Infraestructura_de_clave_p%C3%BAblica)  (http://es.wikipedia.org/wiki/Infraestructura_de_clave_p%C3%BAblica).

2.6.- Principales aplicaciones de la criptografía

Podemos resumir las principales aplicaciones de la criptografía en las siguientes:

- **Seguridad de las comunicaciones.** La criptografía aplicada a las redes de computadores, permite establecer canales seguros sobre redes que no lo son. Además, con la potencia de cálculo actual y empleando algoritmos de cifrado simétrico (que se intercambian usando algoritmos de clave pública) se consigue la privacidad sin perder velocidad en la transferencia.
- **Identificación y autenticación.** Gracias al uso de firmas digitales y otras técnicas criptográficas es posible identificar a un individuo o validar el acceso a un recurso en un entorno de red, con más garantías que con los sistemas de usuario y clave tradicionales.
- **Certificación.** La certificación es un esquema mediante el cual agentes fiables (como una entidad certificadora) validan la identidad de agentes desconocidos (como usuarios reales). El sistema de certificación es la extensión lógica del uso de la criptografía para identificar y autenticar cuando se emplea a gran escala.
- **Comercio electrónico.** Gracias al empleo de canales seguros y a los mecanismos de identificación se posibilita el comercio electrónico, ya que tanto las empresas como los usuarios tienen garantías de que las operaciones no pueden ser espiadas, reduciéndose el riesgo de fraudes y robos.



Q. Patton (CC BY-NC-SA)



Autoevaluación

Una de las principales aplicaciones de la criptografía es la seguridad de las comunicaciones.

☐ Verdadero ☐ Falso

Verdadero

Es verdadero, pues permite establecer canales seguros sobre redes que no lo son.



Para saber más

¿Sabías que **GPG** (GnuPG) es un software incluido últimamente en todas las distribuciones **GNU/Linux** que permite cifrar documentos utilizando una clave privada? Consulta el siguiente enlace, del sitio web Wikipedia, para

ampliar información:

Información sobre GPG  (http://es.wikipedia.org/wiki/GNU_Privacy_Guard).

En este enlace puedes ver un vídeo con información detallada sobre diferentes algoritmos criptográficos:

[https://www.youtube.com/
embed/xSsQwd-2wEc](https://www.youtube.com/embed/xSsQwd-2wEc) ([https://
www.youtube.com/embed/
xSsQwd-2wEc](https://www.youtube.com/embed/xSsQwd-2wEc)).

 EmaUSAC (
Licencia estándar de YouTube)

 Descripción textual del vídeo 

3.- Protocolos seguros de comunicaciones



Caso práctico

Ana está muy orgullosa de comprobar que sus conocimientos sobre criptografía y seguridad de aplicaciones están siendo tenidos en consideración por parte de **Juan**.

Esta tarde, **Ana** ha decidido poner a prueba a su tutor. Se acerca a **Juan** y le pregunta — si ves un candado amarillo junto a la barra de direcciones en tu navegador Web, ¿afirmarías que tienes una conexión segura?. A lo que **Juan** le contesta —aquí no me pillas **Ana**, el candado puede haber sido colocado por un atacante, así que lo mejor es poner la URL directamente en la barra de direcciones utilizando el protocolo seguro HTTPS.



Q Ministerio de Educación (Uso educativo [nc](#))

Si combinamos la criptografía con tecnologías de comunicación en red, entonces hablaremos de protocolos seguros de comunicaciones o protocolos criptográficos.



Debes conocer

En este enlace encontrarás la definición, e información muy detallada, sobre qué es un protocolo criptográfico:

[Concepto de protocolo criptográfico](http://es.wikipedia.org/wiki/Protocolo_criptogr%C3%A1fico) (http://es.wikipedia.org/wiki/Protocolo_criptogr%C3%A1fico)

Algunos de estos protocolos son los siguientes:

- SSL: Proporciona comunicación segura en una conexión cliente/servidor, frente a posibles ataques en la red, como por ejemplo el problema que ya te comentamos al hablar de la criptografía asimétrica, conocido como man in the middle u "hombre en el medio".
- TLS: Es una evolución de SSL, que amplía los algoritmos criptográficos que puede utilizar.

En la imagen puedes ver el nivel, según la pila TCP/IP, en el que se utilizan estos protocolos.

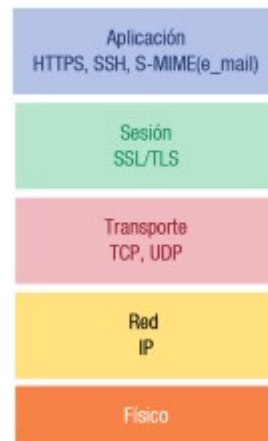
Tanto SSL como TLS son protocolos criptográficos que se ejecutan en una capa intermedia entre un protocolo de aplicación, y un protocolo de transporte como TCP o UDP, por lo que pueden ser utilizados para el cifrado de protocolos de aplicación como Telnet, FTP, SMTP, IMAP o el propio HTTP.

Cuando un protocolo de aplicación, como HTTP o Telnet se ejecuta sobre un protocolo criptográfico como SSL o TLS, se habla de la versión segura de ese protocolo, por ejemplo:

- SSH: Protocolo usado exclusivamente en reemplazo de Telnet, para comunicaciones seguras.
- HTTPS: Protocolo usado exclusivamente para comunicaciones seguras de hipertexto.

Por tanto, podemos decir que el protocolo HTTPS no es ni más ni menos que el protocolo HTTP, pero ejecutándose sobre el protocolo criptográfico SSL, y por ello se dice que HTTPS es un protocolo seguro.

PILA PROTOCOLOS



Q Ministerio de Educación (Uso educativo nc)



Para saber más

En este enlace dispones de una lista de protocolos criptográficos utilizados en la actualidad:

Protocolos criptográficos y estándares  (<https://es.wikipedia.org/wiki/PKCS>).

3.1.- Protocolo criptográfico SSL/TLS

El **protocolo SSL** (Secure Sockets Layer) fue diseñado originalmente por la empresa Netscape para **proporcionar comunicaciones seguras** entre un navegador y un servidor Web, pero se puede utilizar en cualquier otro tipo de **conexión cliente/servidor**. El protocolo **SSL proporciona los siguientes servicios de seguridad**:

- Autenticación.
- Confidencialidad.
- Integridad.

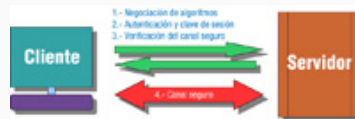
El **protocolo TLS** (Transport Layer Security) **es una evolución del protocolo SSL, una versión avanzada**, que proporciona más algoritmos criptográficos y mayor seguridad frente a nuevos ataques, pero opera de igual forma que SSL.

¿Cómo funciona el protocolo SSL?

Antes de poder garantizar una comunicación segura entre una cliente y un servidor, se deben **acordar o negociar una serie de parámetros en la comunicación en unas fases previas**, es lo que se conoce como acuerdo o handshake (apretón de manos), el SSL/TLS Handshake Protocol. Por otra parte, el protocolo SSL/TLS Record Protocol especifica la forma de encapsular los datos transmitidos y recibidos, incluso los de negociación.

Te indicamos de manera simplificada las **fases que se siguen con SSL**.

1. Fase inicial. Negociar los algoritmos criptográficos a utilizar.



2. Fase de autenticación y clave de sesión. Intercambio de claves y autenticación mediante certificados, utilizando criptografía asimétrica. Se crea la clave para cifrar los datos transmitidos con criptografía simétrica, para agilizar las transmisiones.

Q Ministerio de Educación (Uso educativo nc)

3. Fase fin. Verificación del canal seguro.

4. A partir de aquí, comunicación segura.

Una vez negociados los parámetros de comunicación queda establecido el canal seguro, pero si falla la negociación, la comunicación o canal no se establece.

En su funcionamiento se pueden utilizar entre otros, los siguientes algoritmos:

- Algoritmos de cifrado simétrico: DES, 3DES, RC2, RC4, IDEA.
- Algoritmos de clave pública: RSA.
- Algoritmos de resumen: MD5, SHA.
- Certificados: DSS, RSA.
- Clave de sesión distinta en cada transacción.

Se trata por tanto de un **protocolo de cifrado híbrido**, que utiliza criptografía asimétrica en las fases de negociación y criptografía simétrica en la transmisión de datos.

El éxito del protocolo SSL/TLS se debe fundamentalmente a la expansión del comercio electrónico en Internet, pero también es utilizado para crear redes privadas virtuales VPN.



Para saber más

Te proponemos que visites el siguiente enlace a un vídeo, desarrollado por [intypedia](#), que explica de manera detallada el funcionamiento del protocolo SSL y proporciona aplicaciones como el comercio electrónico y las VPN:

[https://www.youtube.com/
embed/pOeWmStBOYY](https://www.youtube.com/embed/pOeWmStBOYY) ([https://
www.youtube.com/embed/
pOeWmStBOYY](https://www.youtube.com/embed/pOeWmStBOYY))

Q Universidad Politécnica de
Madrid (UPM) (

Licencia estándar de YouTube)

 Descripción textual del vídeo 

3.2.- Otros protocolos seguros

Ya te hemos indicado anteriormente que HTTPS, es la versión segura del protocolo HTTP. La S final indica que es seguro y proviene del uso del protocolo SSL. El protocolo HTTPS es utilizado por entidades bancarias, tiendas en línea y cualquier otro servicio que requiera el envío de información sensible.

HTTPS

Q Ministerio de
Educación (Uso educativo no)

La idea principal de HTTPS es la de crear un canal seguro sobre una red insegura, utilizando cifrado basado en SSL/TLS, el puerto 443 y proporcionando protección frente ataques como el del "hombre en el medio" o man-in-the-middle, siempre que se utilicen métodos de cifrado adecuados y que el certificado del servidor sea verificado y resulte de confianza.

La confianza inherente de HTTPS está basada en una Autoridad de Certificación que viene preinstalada en el software del navegador, de manera que una conexión HTTPS a un sitio web puede ser validada sí y solo si ocurre lo siguiente:

- El usuario confía en la autoridad de certificación.
- El sitio web proporciona un certificado válido.
- El certificado identifica correctamente el sitio web.
- Cada uno de los nodos involucrados en Internet son confiables o el usuario confía en que el cifrado del protocolo SSL/TLS es inquebrantable.

Para saber si la página web que estamos visitando es segura y por tanto utiliza el protocolo HTTPS, debemos observar que en la barra de navegación a la izquierda aparece HTTPS y no HTTP, pues a veces, aunque el propio navegador indique las páginas seguras mediante la imagen de un candado amarillo a la derecha de la barra de direcciones, éste puede haber sido colocado por un atacante.

El protocolo SSH fue diseñado para dar seguridad al acceso a computadores en forma remota.

SSH

Q Ministerio de
Educación (Uso educativo no)

- Cumple la misma función que Telnet, pero utilizando el puerto 22.
- Utiliza criptografía, logrando así seguridad en la conexión.
- Requiere que por parte del servidor exista un demonio que mantenga continuamente en el puerto 22 el servicio de comunicación segura.
- La forma de efectuar su trabajo es muy similar a SSL.
 - El cliente envía una señal al servidor pidiéndole comunicación por el puerto 22.
 - El servidor acepta la comunicación, en el caso de poder mantenerla bajo encriptación mediante un algoritmo definido, y le envía la clave pública al cliente para que pueda descifrar los mensajes.
 - El cliente recibe la clave teniendo la posibilidad de guardar la clave para futuras comunicaciones o destruirla después de la sesión actual.



Autoevaluación

El protocolo HTTPS es la versión segura de HTTP, utilizando el protocolo SSH.

☐ Verdadero ☐ Falso

Falso

Es falso, pues el protocolo criptográfico en el que se basa es en SSL.



Para saber más

A continuación puedes ver un vídeo, con una explicación detallada, sobre el funcionamiento del protocolo SSL y sus fases de conexión:

<https://www.youtube.com/embed/pSVNnShpCOM> (<https://www.youtube.com/embed/pSVNnShpCOM>)

Q Universidad Politécnica de Madrid (UPM)

(Licencia estándar de YouTube)

 Descripción textual del vídeo 

4.- Criptografía en Java

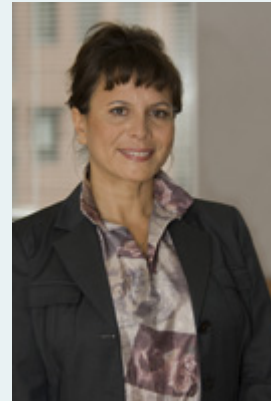


Caso práctico

Ada se ha reunido con **Juan** y **Ana** para ver la marcha del proyecto. Por fin le han dejado la documentación completa y la copia de la aplicación que deben modificar. Como la aplicación no la realizó la empresa BK Programación, han tenido algún que otro problemilla con la obtención de toda la documentación. **Juan** le pregunta a **Ada**: —¿En que lenguaje está desarrollada la aplicación antigua?" — En Java — contesta **Ada**.

Ana sonríe, muestra una cara de total satisfacción, y dice —¡Estupendo!, justo este último trimestre hemos trabajado en clase con las **API** criptográficas de Java.

Ada la observa orgullosa, realmente están muy contentos con el trabajo de **Ana** en la empresa, y le dice —entonces, ya tenéis trabajo ganado —mirando a **Juan** les dice a ambos— ¡Manos a la obra!, que el cliente necesita tener resueltos los problemas de seguridad antes de que finalice este mes.



Q Ministerio de Educación (Uso educativo [nc](#))

Java nos proporciona diferentes **APIs** para la creación de aplicaciones con comunicaciones seguras. Con estas APIs podremos realizar, entre otras las siguientes tareas:

- Encriptación de la información con clave pública y privada.
- Firma digital y su verificación.
- Resúmenes de mensajes (HASH).
- Certificados digitales y validación de certificados.
- Comunicaciones de red seguras con el protocolo SSL.



Q Ministerio de Educación (Uso educativo [nc](#))

Además, estas APIs permiten utilizar diferentes algoritmos criptográficos (DES, RSA, MD5, etc.), proporcionados por lo que Java denomina proveedor de servicios criptográficos (PSC).

La arquitectura criptográfica de Java fue introducida en el JDK 1.1, incluyendo bibliotecas de acceso a funciones criptográficas de propósito general, conocidas como **Arquitectura Criptográfica de Java (JCA)**, y otras utilidades criptográficas que complementan a las primeras y que se conocen como **Extension Criptográfica de Java (JCE)**.

A partir de JDK 1.4, JCE viene incluido en el JDK y la distinción entre ambos

componentes cada vez es menos evidente, por lo que podemos decir que JCA es el core o corazón del API de seguridad en Java y que JCE es una parte de la JCA.

Los paquetes que conforman la **arquitectura criptográfica JCA y JCE** son:

- Paquete `java.security`
- Paquete `javax.crypto`

Otras bibliotecas disponibles en el JDK, y que utilizan la arquitectura proveedor de JCA, son las que proporcionan:

- **Comunicaciones seguras.**

- Extensión Java Sockets Seguros.(JSSE). `javax.net.ssl.SSLSocket`
- Servicio General de Seguridad a través de Kerberos. (JGSS).
- Capa de autenticación y seguridad Java (SASL).
`javax.security.sasl.Sasl`

- **Autenticación y control de Acceso.**

- Servicio de Autenticación y Autorización. (JAAS).
`javax.security.auth.login`.



Para saber más

En la web oficial de Java puedes consultar todos los paquetes que se proporcionan para gestionar la seguridad en aplicaciones con comunicaciones:

 Información sobre los paquetes Java para gestionar la seguridad 
(<https://docs.oracle.com/en/java/javase/24/docs/api/java.security.sasl/javax/security/sasl/package-summary.html>).

4.1.- Arquitectura criptográfica de Java

La **arquitectura criptográfica de Java**, denominada **JCA**, es la pieza principal de la criptografía de Java y está diseñada en torno a los siguientes principios:

- **Independencia de la aplicación.**

Los programas no necesitan implementar algoritmos criptográficos, sino que los utilizan a través de la plataforma Java, mediante su solicitud a un PSC. Además, se pueden utilizar varios proveedores en la misma aplicación.



Q Ministerio de Educación (Uso educativo n.c.)

- **Interoperabilidad.** Los diferentes proveedores son compatibles con todas las aplicaciones, esto es, una aplicación concreta no tiene por que estar ligada a un proveedor específico, ni un proveedor estará ligado a una aplicación específica.

- **Extensibilidad.** La plataforma Java incluye una serie de proveedores integrados que implementan un conjunto básico de servicios de seguridad utilizados ampliamente en la actualidad, pero también permite la instalación de proveedores personalizados que implementan nuevos servicios.

La **arquitectura JCA**, emplea un modelo basado en el uso de proveedores, de manera que se puede decir que incluye dos tipos de componentes:

- Bibliotecas de clases e interfaces: paquete `java.security`.
- Proveedores de servicios criptográficos que proporcionan las implementaciones de diferentes algoritmos criptográficos, como por ejemplo `Sun`.

La **extensión criptográfica de Java (JCE)**, emplea el mismo modelo basado en proveedores que JCA, en este caso está presente mediante el proveedor `SunJCE` y proporciona una serie de servicios que permiten complementar los servicios de seguridad de JCA, mediante los paquetes `javax.crypto`, `javax.crypto.spec`, `javax.crypto.interfaces`.

Ya te hemos comentado que a partir de JDK 1.4, la distinción entre JCA y JCE es menos evidente, al ir incluido JCE en le propio JDK, de manera que se habla de estos componentes de manera conjunta. Por tanto, podemos decir que **la arquitectura criptográfica de Java incluye los siguientes componentes software**:

- Las **bibliotecas de clases e interfaces** que proporcionan las diferentes funcionalidades de seguridad, como son:
 - `java.security`. Consiste básicamente en clases abstractas e interfaces que proporcionan el manejo de certificados, claves, resúmenes de mensajes y firmas digitales.
 - `javax.crypto`. Proporciona las clases e interfaces para realizar operaciones criptográficas como encriptación/desencriptación,

generación de claves y acuerdo de claves, así como generación de códigos de autenticación de mensajes (MAC).

- `javax.crypto.spec`. Incluye varias clases de especificación de claves y de parámetros de algoritmos.
 - `javax.crypto.interfaces`. presenta las interfaces de las claves empleadas en los algoritmos de tipo Diffie-Hellman (clases `DHKey`, `DHPrivateKey` y `DHPublicKey`).
- Los **proveedores de servicios criptográficos**, tales como `Sun`, `SunRsaSign`, `SunJCE`, que contienen las implementaciones de diferentes algoritmos criptográficos.

Lo que se conoce como **Arquitectura de Proveedores** es lo que permite que coexistan múltiples implementaciones de algoritmos criptográficos.



Autoevaluación

La arquitectura criptográfica de Java está diseñada de manera que siempre se deben utilizar los mismos algoritmos criptográficos.

☐ Verdadero ☐ Falso

Falso

Es falso, pues gracias a la propiedad de extensibilidad, se pueden incluir nuevos proveedores con nuevos algoritmos.



Citas para pensar

"Sorprenderse, extrañarse, es comenzar a entender".

José Ortega y Gasset.

4.2.- Proveedores y motores criptográficos

Un **proveedor de servicios criptográficos** no es ni más ni menos que una empresa de seguridad que genera sus propios servicios de seguridad en Java, implementando las clases e interfaces definidos en JCA/JCE y siguiendo el estándar definido en JCA.



🔍 Motores criptográficos en Java (Copyright)

Un **motor criptográfico** (engine) es el conjunto de clases e interfaces que debe implementar un proveedor de servicios criptográficos. Algunas de ellas son las siguientes:

- `MessageDigest`. Para resúmenes de mensajes.
- `Signature`. Para firmas digitales.
- `KeyFactory`. Factoría para el manejo de claves seguras.
- `KeyPairGenerator`. Para generar claves públicas y privadas.
- `Cipher`. Para encriptación y desencriptación de información.

Cada una de estas clases incluyen un método `getInstance()` que devuelve un algoritmo criptográfico de un proveedor dado.

Un proveedor por defecto implementado dentro de JDK es el proveedor de nombre SUN que proporciona:

- Implementación de DSA, MD5 y SHA-1.
- Generación de claves públicas y privadas para el algoritmo DSA.
- Factoría de claves que soportan conversión de claves públicas a privadas.
- Construcción de certificados X.509.

Así por ejemplo:

- `MessageDigest.getInstance("MD5");` Obtiene el algoritmo MD5 del proveedor por defecto de más prioridad.
- `MessageDigest.getInstance("MD5", "ProveedorA");` Obtiene el algoritmo MD5 del proveedor ProveedorA.

Y ¿cómo **podemos utilizar más de un proveedor criptográfico en nuestra aplicación**? Para ello solo tienes que modificar el fichero `java.security` situado en el directorio `lib/security`. El formato de definición de cada uno de los proveedores es el siguiente:

- `security.provider.n=nombre_de_la_clase`.



Autoevaluación

En una aplicación java solo se puede utilizar un proveedor de servicios criptográficos.

☐ Verdadero ☐ Falso

Falso

Es falso, se pueden utilizar más de uno, modificando el fichero `java.security`.



Para saber más

En el siguiente enlace puedes profundizar en los conceptos relacionados con la arquitectura Java (motor y proveedor) y ver un programa Java que permite listar los proveedores disponibles en tu equipo:

 Documento sobre conceptos y ejemplos respecto a JCA  (<https://ccia.esei.uvigo.es/docencia/SSI/2223/practicas/tutorial-jca.pdf>) (0.14 MB)

Desde este otro enlace puedes consultar en detalle todas las clases e interfaces que proporciona la Arquitectura Criptográfica de Java, así cómo la forma en que se gestionan los proveedores:

Guía oficial de referencia sobre JCA/JCE  (<https://docs.oracle.com/en/java/javase/24/security/java-cryptography-architecture-jca-reference-guide.html#GUID-EAB9FF73-4C69-4FD5-8A0C-5CF48211A859>)

4.3.- Gestión de claves con el paquete Java.security

En la generación de claves se utilizan números aleatorios seguros, que son números aleatorios que se generan en base a una semilla. Esto permite crear algoritmos seguros, pues será muy difícil determinar los valores generados sin conocer la semilla.

```
package os.ncdd.cidoac.dam.psp.07.psp07gestionclaves;

import java.security.Key;
import java.security.KeyPairGenerator;
import java.security.SecureRandom;
import javax.crypto.KeyGenerator;

/**
 * @author: Alejandro Loal
 */
public class Main {

    public static void main(String[] args) {

        Key key;
        KeyPairGenerator keyPairGenerator;
        SecureRandom secureRandom;
        KeyGenerator keyGenerator;

    }
}
```

El paquete `java.security` proporciona las siguientes clases para la gestión de claves:

Q Ministerio de Educación (Uso educativo no)

- El **interface** `Key`, permite la representación de claves, su almacenamiento y envío de forma serializada dentro de un sistema. Se trata de un **interface** `Serializable` y proporciona entre otros los siguientes métodos:
 - `getAlgorithm()`. Devuelve el nombre del algoritmo con el que se ha generado la clave (RSA, DES, etc.).
 - `getEncoded()`. Devuelve la clave como un array de bytes.
 - `getFormat()`. Devuelve el formato con el que está codificada la clave.
- La **clase** `KeyPairGenerator` permite la generación de claves públicas y privadas (asimétricas). Genera objetos del tipo `KeyPair`, que a su vez contienen un objeto del tipo `PublicKey` y otro del tipo `PrivateKey`.
 - El método `initialize()` permite establecer el tamaño de la clave y el número aleatorio a partir del cual será generada.
- La **clase** `KeyGenerator` permite la generación de claves privadas (simétricas). Genera objetos de tipo `SecretKey`.
 - El método `init()` permite establecer el tamaño de la clave y el número aleatorio a partir del cual será generada.
- La **clase** `SecureRandom` permite generar números aleatorios seguros.
 - El método `setSeed()` permite establecer el valor de la semilla.
 - El constructor `secureRandom()` utiliza la semilla del proveedor SUN.
 - El método `next()` y el `nextBytes()` obtienen el valor de los números generados.

La creación de claves se basa en el tamaño de las mismas, de manera que, si incrementas mucho el tamaño de una clave, el tiempo de cálculo de la misma también se incrementará; y esto puede suponer que la administración de claves no sea lo suficientemente ágil para una determinada aplicación.

Desde el siguiente enlace puedes descargar un proyecto java que genera una pareja de claves (pública y privada) mediante la clase `KeyPairGenerator` y muestra los valores obtenidos de la `PrivateKey` y la `PublicKey`:

 Proyecto Java, desarrollado en NetBeans, que genera un par de claves (privada/pública) haciendo uso de 'KeyPair' 

(3.6 KB)

En el siguiente enlace puedes ver otro ejemplo de generación de claves en el que se indica además del tamaño de la clave, el número aleatorio seguro a partir del que se genera:

 Proyecto Java, desarrollado en NetBeans, que genera un par de claves indicando el valor seguro a partir del cual se realiza 

(4.4 KB)



Autoevaluación

El método `initialize()` establece el valor de la semilla al generar números aleatorios.

☐ Verdadero ☐ Falso

Falso

Es falso, lo que establece es el tamaño de la clave.



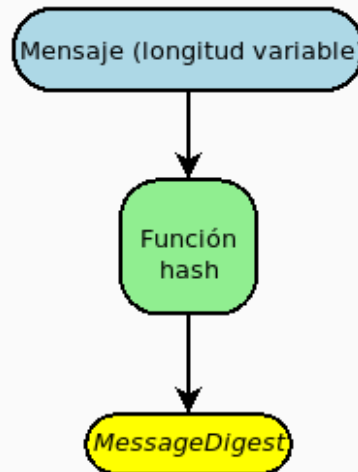
Para saber más

Desde este enlace, puedes consultar todas las interfaces y clases del paquete `java.security` en la web oficial:

Interfaces y clases del paquete `Java.security`  (<https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/security/package-summary.html#>).

4.4.- Resúmenes de mensajes con la clase MessageDigest

La clase `MessageDigest` del paquete `java.security` proporciona la funcionalidad del algoritmo de resumen de mensajes de una aplicación, empleando para ello el algoritmo y proveedor especificados. El resumen del mensaje es una función hash unidireccional segura que acepta datos de cualquier tamaño y genera un valor [hash](#) de longitud fija. Los métodos que debes utilizar para crear un resumen de mensaje son:



Q Ministerio de Educación (Uso educativo nc)

- `getInstance()` : obtiene el algoritmo de resumen.
- `update()` : obtiene el resumen.
- `digest()` : completa la obtención del resumen.

En JDK podemos encontrar dos **algoritmos de resumen de mensajes**:

- MD5. Genera una salida de 128 bits de longitud fija.
- SHA-1. Genera una salida de 160 bits.

El siguiente código es un ejemplo sencillo de creación de un resumen de mensaje con algoritmo SHA-1:

 Proyecto Java, desarrollado en NetBeans, que genera un mensaje resumen, empleando el algoritmo 'SHA-1', a partir de un texto de origen 

(3.3 KB)



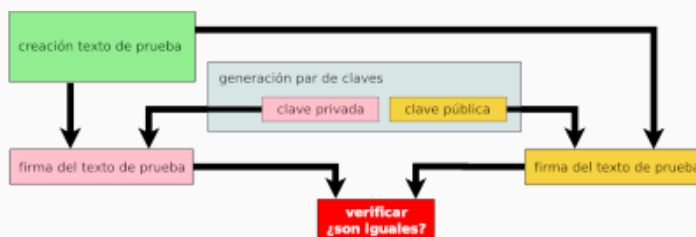
Citas para pensar

"No entiendes realmente algo a menos que seas capaz de explicárselo a tu abuela"

Albert Einstein.

4.5.- Firma digital con la clase Signature

La clase `Signature` del paquete `java.security` permite realizar una firma digital así como hacer su verificación.



Q Ministerio de Educación (Uso educativo nc)

Los pasos que debes seguir para realizar la firma de un mensaje y después verificarla son los siguientes:

- Generar las claves públicas y privadas mediante la clase `KeyPairGenerator`:
 - La `PrivateKey` la utilizaremos para firmar.
 - La `PublicKey` la utilizaremos para verificar la firma.
- Realizar la firma digital mediante la clase `Signature` y un algoritmo asimétrico, por ejemplo `DSA`.
 - Crearemos un objeto `Signature`.
 - Al método `initSign()` le pasamos la clave privada.
 - El método `update()` creará el resumen de mensaje.
 - El método `sign()` devolverá la firma digital.
- Verificar la firma creada mediante la clave pública generada.
 - Al método `initVerify()` le pasaremos la clave pública.
 - Con `update()` se actualiza el resumen de mensaje para comprobar si coincide con el enviado.
 - El método `verify()` realizará la verificación de la firma.

El algoritmo de firma digital **EdDSA** viene implementado en las librerías de Java, es parte del estándar de firmas digitales DSS, y con él se pueden utilizar los algoritmos de resumen MD5 y SHA-1. Es el que utilizaremos en nuestro ejemplo.

En este enlace encontrarás un proyecto Java completo que firma digitalmente un texto y verifica su firma digital:

 [Proyecto Java, desarrollado en NetBeans, que genera un mensaje resumen, empleando el algoritmo 'SHA-1', a partir de un texto de origen](#) 

(4.6 KB)



Autoevaluación

Señala las opciones correctas. Para realizar una firma digital con `Signature`:

- ☐ Hay que pasar al método `initVerify()` la clave pública.
- ☐ Al método `initSign()` se le pasa la clave privada.
- ☐ Hay que actualizar con `update()` el resumen del mensaje.
- ☐ La firma digital la devuelve el método `sign()`.

Mostrar retroalimentación

Solución

1. Incorrecto (#answer-59_58)
2. Correcto (#answer-59_126)
3. Incorrecto (#answer-59_128)
4. Correcto (#answer-59_130)



Para saber más

Desde la versión 6 de Java es posible realizar firmas digitales en documentos XML. En el siguiente enlace encontrarás ejemplos de cómo poder realizar esto:

Firma digital con Java de documentos XML  (<http://www.adictosaltrabajo.com/tutoriales/tutoriales.php?pagina=xmlSignature>)

- ☐ Siempre realiza encriptación de bloques.
- ☐ Admite varios modos de operación.

No es correcto, prueba de nuevo.

No es correcto, deberías haber leído mejor.

No es correcto, debes prestar más atención.

Correcto. Muy bien, vamos por buen camino.

Solución

1. Incorrecto (#answer-62_63)
2. Incorrecto (#answer-62_135)
3. Incorrecto (#answer-62_138)
4. Opción correcta (#answer-62_141)



Para saber más

En el siguiente enlace puedes consultar la documentación oficial de las clases e interfaces del paquete `javax.crypto`:

 Documentación oficial del paquete `javax.crypto`  (<https://docs.oracle.com/en/java/javase/24/docs/api/java.base/javax/crypto/package-summary.html>)

4.6.1.- Ejemplos de encriptación simétrica

Te hemos hablado del uso de la encriptación para garantizar la confidencialidad en el envío de mensajes, pero también podemos utilizarla para proteger información almacenada.

¿Cómo podemos cifrar un archivo almacenado en disco? Este es el ejemplo que precisamente vamos a ver a continuación.



Q Encriptación datos (CC BY-NC-SA)

En el siguiente recurso didáctico puedes ver un ejemplo sencillo para ilustrar la encriptación y desencriptación de un archivo mediante clave privada, utilizando el algoritmo DES:

**Cifrado de un fichero
con clave privada
DES**

Preparar fichero a encriptar

- Creamos un archivo de nombre **archivo** con algún texto
- Lo situamos en la ruta **/home/usuario**

/home/usuario

crypto
archivo

Este texto será encriptado y
desencriptado mediante
clave privada y
algoritmo DES

Paquetes, clases y métodos

- Las clases necesarias están en las librerías siguientes:

```
import java.security.*;  
import javax.crypto.*;  
import java.io.*;
```

- Para cifrar/descifrar el archivo utilizaremos un objeto de la clase :

```
KeyGenerator.getInstance("DES")  
    .init(56)  
    .generateKey()
```

- Creación de clave

```
Cipher.getInstance("DES")  
    .init(Cipher.ENCRYPT_MODE, key)  
    .doFinal(textoPlano)
```

- Instancia para
cifrado de archivos

```
Cipher.getInstance("DES")  
    .init(Cipher.DECRYPT_MODE, key)  
    .doFinal(textoCifrado)
```

- Instancia para
descifrado de
archivos

Método *main()*

- El método `main()`
 - Invoca a un método para obtener la clave: `preparacion(...)`
 - Invoca a un método para cifrar: `encriptarArchivo(...)`
 - Invoca a un método para descifrar: `descifrarArchivo(...)`

```
public static void main(String[] args) {  
    //declara e inicializa el objeto tipo 'clave secreta':  
    → SecretKey clave = preparacion("DES",56);  
  
    // ahora se realiza la invocación de métodos para  
    // encriptación/desencriptación de archivos  
  
    → // encriptación (pasar 'texto claro' a texto no legible)  
    encriptarArchivo(    "/home/usuario/archivo",  
                        "/home/usuario/archivo.cifrado",  
                        clave);  
  
    → // desencriptación o descifrado (pasar texto no legible a 'texto claro')  
    descifrarArchivo("/home/usuario/archivo.cifrado",  
                     "/home/usuario/archivo.descifrado",  
                     clave);  
}
```

Método *preparacion(...)*

Se le pasa como parámetros el algoritmo y la longitud y retorna la clave para cifrar/descifrar

```
private static SecretKey preparacion(String algoritmo, int longitud) {  
    KeyGenerator keyGen;  
    SecretKey clave = null;  
  
    try{  
        //1. Crear e inicializar clave  
        System.out.print("1.- Genera clave DES");  
        //crea un objeto para generar la clave usando algoritmo 'DES':  
        keyGen = KeyGenerator.getInstance(algoritmo);  
        keyGen.init(longitud); //se indica el tamaño de la clave en bytes  
        clave = keyGen.generateKey(); //genera la clave privada  
  
        System.out.print(" [ Clave: ");  
        // muestra la clave ( en un entorno real )  
        mostrarBytes(clave.getEncoded());  
        System.out.print(" ]\n");  
    } catch (NoSuchAlgorithmException e) {  
        System.out.println("Error: " + e.getMessage());  
    } finally{  
        return clave;  
    }  
}
```

La primera tarea de este método será crear la clave secreta para el algoritmo indicado, con un tamaño determinado (en bytes)

Método *encriptarArchivo(...)*

Se le pasa como parámetros el fichero a cifrar, el archivo donde grabar el resultado y la clave para cifrar

La primera tarea de este método será configurar el 'cifrador' en 'modo encriptación' empleando la clave proporcionada

```
private static void encriptarArchivo(String archivo,
                                     String archivoEncriptado,
                                     SecretKey clave) {

    Cipher cifrador;

    try {
        // creación de objeto Cipher para cifrar (algoritmo 'DES')
        cifrador = Cipher.getInstance("DES");

        //2. Establecer el cifrador en modo CIFRADO o ENCRYPTACIÓN
        cifrador.init(Cipher.ENCRYPT_MODE, clave);

        System.out.println("2.- Encriptación con algoritmo 'DES' del archivo"
                           + " fuente '"
                           + archivo
                           + "'. Se almacena el resultado en '"
                           + archivoEncriptado
                           + "'");

        procesarArchivos(archivo, archivoEncriptado, cifrador);
    } catch (InvalidKeyException |
             NoSuchAlgorithmException | NoSuchPaddingException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
```

Tras lo anterior procedemos a invocar al método para procesar el archivo fuente y generar el resultado

Método *desencriptarArchivo(...)*

Se le pasa como parámetros el fichero a descifrar, el archivo donde grabar el resultado y la clave para cifrar

La primera tarea de este método será configurar el 'cifrador' en 'modo desencriptación' empleando la clave proporcionada

Tras lo anterior procedemos a invocar al método para procesar el archivo fuente y generar el resultado

```
private static void desencriptarArchivo(String archivoEncriptado,
                                         String archivoDesencriptado,
                                         SecretKey clave) {

    try {
        // creación de objeto Cipher para descifrar (algoritmo 'DES')
        Cipher cifrador = Cipher.getInstance("DES");

        //3.- Configurar cifrador en modo DESCIFRADO o DESENCRIPTACIÓN:
        cifrador.init(Cipher.DECRYPT_MODE, clave);

        System.out.println("3.- Desencriptación con algoritmo 'DES' del"
                           + " archivo fuente '"
                           + archivoEncriptado
                           + "'. Se almacena el resultado en '"
                           + archivoDesencriptado
                           + "'");

        procesarArchivos(archivoEncriptado, archivoDesencriptado, cifrador);
    } catch (InvalidKeyException |
             NoSuchAlgorithmException | NoSuchPaddingException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
```

Método *procesarArchivos(...)* (I)

Se le pasa como parámetros el fichero a cifrar/descifrar, el archivo donde grabar el resultado y el objeto para cifrar

```
private static void procesarArchivos(String entrada,
                                       String salida,
                                       Cipher cifrador) {
    //archivo que contiene la información que debe ser manipulada:
    FileInputStream archivoEntrada;
    //archivo donde se almacena la información una vez es procesada:
    FileOutputStream archivoSalida;

    // controla los bytes que se leen desde el
    // archivo fuente
    int bytesLeídos;

    try {
        archivoEntrada = // descriptor de archivo 'fuente':
            new FileInputStream(entrada);
        archivoSalida = // descriptor de archivo 'destino':
            new FileOutputStream(salida);

        byte[] bufferTextoProcesado;
        byte[] buffer = new byte[1000]; //array de bytes

        // lee el archivo de 1000 en 1000 bytes e invoca al cifrador
        // con los fragmentos leídos:
        bytesLeídos = archivoEntrada.read(buffer, 0, 1000);
    }
```

Tras lo anterior procedemos a invocar a leer, en bloques de 1000 bytes, los datos del archivo fuente

La primera tarea de este método será obtener los descriptors de los archivos (fuente y destino)

Método *procesarArchivos(...)* (II)

En cada iteración se realiza la operación de cifrado/descifrado con los bytes leídos del archivo fuente

```
while (bytesLeídos != -1) { //mientras no se llegue al final del
    // archivo pasa texto cifrado ('bytesLeídos') a 'cifrador' y
    // lo descifra, asignándolo a 'bufferTextoProcesado':
    bufferTextoProcesado = cifrador.update(buffer, 0, bytesLeídos);
    archivoSalida.write(bufferTextoProcesado); // graba el resultado
    bytesLeídos = archivoEntrada.read(buffer, 0, 1000);
}

// finaliza el proceso de encriptación/desencriptación:
bufferTextoProcesado = cifrador.doFinal();

// escribe el final del texto 'cifrado'/'no cifrado' si lo hubiera:
archivoSalida.write(bufferTextoProcesado);

// cierra los archivos y libera los recursos asociados:
archivoEntrada.close();
archivoSalida.close();
} catch (IOException | BadPaddingException | IllegalBlockSizeException e) {
    System.out.println("Error: " + e.getMessage());
}
```

Tras lo anterior procedemos a escribir los datos, tras haberlos transformados, en el archivo destino

Se finaliza el procesamiento de los datos

Método `mostrarBytes(...)`

```
/**
 * Método que muestra los bytes contenidos en un array
 * @param buffer array con la información
 */
public static void mostrarBytes(byte[] buffer) {
    System.out.write(buffer, 0, buffer.length);
}
```

Vuelca en el buffer de salida del sistema el contenido del array de bytes que se le pasa como argumento

Resultado de la ejecución

/home/usuario/archivo

Este texto será encriptado y
desencriptado mediante clave
privada y algoritmo DES

Archivo original

/home/usuario/archivo.cifrado

©_oQÅJ÷\$;PÔ«Ž
i2CÔÁpiv`s0%tn+¢2CÔÁpivCð/EnF
çEZ°~KYÔ4R‡(5~â>□,)XS"—

Archivo obtenido
tras el proceso
de encriptación

/home/usuario/archivo.descifrado

Este texto será encriptado y
desencriptado mediante clave
privada y algoritmo DES

Archivo obtenido
tras el proceso de
desencriptación

Credenciales

Imagen	Datos de licencia
Todas las capturas de pantalla de este presentación, salvo la diapositiva 2 tienen como datos de licencia: Autoría: Alejandro Leal Cruz Licencia: Uso educativo-no comercial. Procedencia: Captura de pantalla del programa y editor de código NetBeans, propiedad the The Apache Software Foundation, bajo licencia Apache License v.2.	
	Autoría: Alejandro Leal Cruz Licencia: Uso educativo-no comercial. Procedencia: esquema realizado por el autor.

En este enlace dispones del proyecto Java completo. Observa que, al ejecutar el programa, se muestra la clave generada (con fines didácticos):

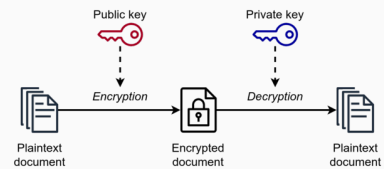
 [Proyecto Java de encriptación mediante clave simétrica](#)  (5.8 KB)

En el siguiente enlace puedes ver otro ejemplo de cifrado simétrico de un fichero. En este caso el programa solicita la operación a realizar, cifrar o descifrar, y el archivo que debe ser procesado:

[Sitio web con ejemplo de cifrado de un archivo](#) 

4.6.2.- Ejemplo de encriptación asimétrica

En este apartado vamos a trabajar un ejemplo de cifrado, se proporciona el proyecto completo, mediante cifrado asimétrico de clave pública (RSA). Puedes descargarlo en el siguiente enlace:



Q MarcT0K (CC BY)

 [Proyecto Java de encriptación asimétrica \(clave pública, algoritmo RSA\)](#)  (4.8 KB)



Citas para pensar

"Aprender es como remar contra corriente: en cuanto se deja, se retrocede".

Edward Benjamin Britten.



Para saber más

Consulta el siguiente enlace para obtener más información sobre la clase **Cipher**:

 Clase Cipher  (<https://docs.oracle.com/en/java/javase/24/docs/api/java.base/javax/crypto/Cipher.html>)

5.- Sockets seguros en Java (JSSE)



Caso práctico

Aunque les ha costado varios quebraderos de cabeza, **Ana** y **Juan** llevan muy avanzado su trabajo. Hoy han comenzado a revisar todo el tema de sockets y a hacer pruebas de conexión. **Ana** acaba de verlo claro, se acerca a **Juan** y le dice —¡observa esta línea del programa!, aquí hay un fallo de seguridad pues no se están utilizando sockets seguros.

Juan asiente —Así es, y por eso fue tan fácil interceptar el tráfico entre servidor y clientes. —Para terminar diciendo— al final la aplicación no era tan segura como nos habían dicho —y le hace un guiño **Ana**.



Q Ministerio de Educación. (Uso educativo nc)

Otra biblioteca disponible e integrada en Java desde el JDK 1.4 es JSSE (Extensión Java Sockets Seguros):



- Utiliza la arquitectura proveedor de JCA.

Q Ministerio de Educación (Uso educativo nc)

- Proporciona **comunicaciones seguras mediante el protocolo SSL** (Secure Sockets Layer) soportado a través del paquete `javax.net.ssl`.

En unidades anteriores, has estudiado y visto ejemplos de cómo la programación de sockets proporciona un mecanismo de muy bajo nivel para la comunicación e intercambio de datos entre dos ordenadores, uno considerado como cliente, que es el que inicia la conexión con el otro, el servidor, que está a la espera de conexiones de clientes.

Un **socket seguro** es un **socket** basado en el protocolo SSL y por tanto proporcionará en una comunicación autenticación, integridad y confidencialidad de los datos transmitidos.

Los usos más habituales de SSL en Java se basan en las siguientes clases:

- `SSLSocket`, para programar **sockets** seguros de cliente.
- `SSLServerSocket`, para programar **sockets** seguros de servidor.

Los **sockets seguros** son parecidos en su funcionamiento a los **sockets** normales, pero con ciertos cambios en su proceso de creación e inicialización.



Para saber más

Si necesitas un repaso de los conceptos relacionados con la programación

de **sockets** en Java, te recomendamos que visites el siguiente enlace, que además te proporciona varios ejemplos:

 [Programación de sockets en Java](https://www.infor.uva.es/~fdiaz/sd/doc/java.net.pdf)  (<https://www.infor.uva.es/~fdiaz/sd/doc/java.net.pdf>)

En este otro enlace, dispones del tutorial, en inglés, sobre JSSE, donde puedes consultar el funcionamiento de SSL y todas las clases Java relacionadas con SSL:

 [Tutorial java sobre JSSE](https://docs.oracle.com/en/java/javase/24/security/java-secure-socket-extension-jsse-reference-guide.html)  (<https://docs.oracle.com/en/java/javase/24/security/java-secure-socket-extension-jsse-reference-guide.html>)

5.1.- Desarrollar socket seguro (lado servidor)

Para crear `sockets` seguros de servidor, Java utiliza el patrón de diseño `Factory`, de manera que lo primero que haremos siempre es obtener un objeto `SSLServerSocketFactory`. Este objeto encapsula los detalles de creación, configuración e inicialización del `socket` seguro, como es la autenticación de claves, validación de certificados, etc.

SSLServerSocket

Q Ministerio de Educación (Uso educativo nc)

Después crearemos un objeto `SSLServerSocket`. Esta clase soporta todos los métodos estándar de la clase `ServerSocket`, además de métodos específicos para trabajar con aspectos de seguridad.

Podemos resumir los **pasos para programar un `socket` seguro de servidor** en los siguientes:

- Obtener un objeto `SSLServerSocketFactory`.
- Crear un objeto `SSLServerSocket` indicando el puerto de escucha del servidor.
- Crear un `socket` seguro cliente que esté atento a las posibles conexiones al servidor.
- Crear un canal seguro sobre el `socket` abierto.

En el siguiente segmento de código, puedes ver los pasos indicados anteriormente, donde el `socket` seguro del servidor escucha por el puerto 5000:

```
// Declaración de objeto de la clase SSLServerSocketFactory para crear
SSLServerSocketFactory factoria // socket SSL en el lado servidor servidor:
= (SSLServerSocketFactory) SSLServerSocketFactory.getDefault();

SSLServerSocket socketServidorSSL = null;
BufferedReader entrada = null;

try{
    // creación de un socket servidor seguro:
    socketServidorSSL
    = (SSLServerSocket) factoria.createServerSocket(5000);
    SSLServerSocket socketSSL = (SSLServerSocket) socketServidorSSL.accept();
    // creación de canal seguro en el socket anteriormente establecido:
    entrada = new BufferedReader(
        new InputStreamReader(socketSSL.getInputStream()));
    // el resto de operaciones se realizan con el objeto 'entrada':
    // ...
    // ...
}
```

Q Ministerio de Educación (Uso educativo nc)

 **Bloque de código Java para crear un socket seguro desde el lado del servidor**



(1.5 KB)



Autoevaluación

Para crear un socket seguro de servidor, primero se crea un objeto `SSLServerSocket` y después un objeto `SSLServerSocketFactory`.

☐ Verdadero ☐ Falso

Falso

Es falso, justo es al contrario.



Para saber más

En este enlace dispones de la documentación oficial sobre la clase `SSLServerSocket`:

 Información sobre la clase `SSLServerSocket`  (<https://docs.oracle.com/en/java/javase/24/docs/api/java.base/javax/net/ssl/SSLServerSocket.html>)

5.2.- Desarrollar socket seguro (lado cliente)

Para crear un `socket` seguro cliente se puede utilizar un objeto `SSLSocketFactory`, aunque no siempre es necesario, pues también puede crearse en el momento de aceptar una conexión del servidor en obligado o in-bound connection.

SSLSocket

Q Ministerio de Educación (Uso educativo [nc](#))

Después se crearemos un objeto `SSLSocket`. Esta clase soporta todos los métodos estándar de la clase `Socket`, además de métodos específicos para trabajar con `sockets` seguros.

En resumen, los pasos para programar un `socket` seguro cliente pueden ser:

- Obtener un objeto `SSLSocketFactory`.
- Crear un objeto `SSLSocket` indicando el nombre del servidor y puerto de escucha.
- Crear un canal seguro de comunicación con el servidor.

En el siguiente segmento de código puedes ver los pasos indicados para crear un `socket` cliente seguro, que se conectará al servidor de nombre `localhost` y por el puerto 5000:

```
// Declaración de objeto de la clase
SSLSocketFactory factoria // factoria para crear sockets SSL
= (SSLSocketFactory) SSLSocketFactory.getDefault();

// variables de las clases para las comunicaciones:
SSLSocket socketSSL = null;
BufferedReader entrada = null;
BufferedWriter salida = null;

try {
    // creación de un socket seguro:
    socketSSL = (SSLSocket) factoria.createSocket("localhost", 5000);

    // se comunica con la consola (entrada del usuario):
    entrada = new BufferedReader(
        new InputStreamReader(System.in));

    // el resto de operaciones se realizan con el objeto 'entrada':
    // ...
    // ...
}
```

Q Ministerio de Educación (Uso educativo [nc](#))

 [Bloque de código Java para crear un socket seguro desde el lado del cliente](#) 

(2 KB)



Autoevaluación

Para programar un socket seguro cliente no siempre es necesaria crear un objeto `SSLSocket`.

☐ Verdadero ☐ Falso

Falso

Es falso, todos los `sockets` seguros cliente son objetos `SSLSocket`.



En este enlace dispones de la documentación oficial sobre la clase `SSLSocket`:

 Información sobre la clase `SSLSocket`  (<https://docs.oracle.com/en/java/javase/24/docs/api/java.base/javax/net/ssl/SSLSocket.html>)

5.3.- Ejemplos de aplicaciones con comunicaciones seguras

Puesto que SSL utiliza certificados digitales para la autenticación, necesitamos crear el certificado para nuestro servidor. JSSE puede utilizar los certificados creados por la herramienta keytool de Java. De esta forma, la creación de los certificados la podemos hacer con la ejecución de keytool y con los parámetros pertinentes.

En el siguiente recurso didáctico puedes ver un ejemplo de uso de keytool para esta finalidad:



Q Ministerio de Educación (Uso educativo nc)

**Herramienta
keytool**

Descripción de Keytool

- **Keytool** es una utilidad que permite la gestión de
 - Claves
 - Certificados
- **Keytool** permite a los usuarios:
 - Administrar pares de claves pública/privada y los certificados asociados para su uso en una autenticación.
 - Almacenar en caché las claves públicas, en forma de certificados, de las personas con las que se comunican.
- **Keytool** permite almacenar claves y certificados en un almacén denominado **Keystore**(almacén de certificados).

Keystore y Truststore

- **Keystore:** almacén de certificados. Desde la perspectiva de JSSE es una base de datos de los pares llaves y de los certificados que se utilizan para la autenticación del SSL.
- **Truststore:** almacena certificados de las Autoridades de Certificación (CA). Se utilizan para verificar las identidades de otros clientes y servidores.
 - Cuando un cliente o un servidor inicia una sesión del SSL, extrae sus certificados y claves de su almacén de certificados (keystore).
 - Cuando verifica las identidades de otros clientes o servidores, extraerá los certificados de la Autoridad de la Certificación de su truststore.

Creación de un certificado para utilizar con SSL (I)

El siguiente comando crea un certificado de clave pública de nombre **claveSSL**, con el algoritmo **RSA** y lo almacena en el keystore de nombre **AlmacenSSL**

```
keytool -genkey -alias claveSSL -keyalg RSA -keystore AlmacenSSL
```

Creación de un certificado para utilizar con SSL (II)

Al introducir el comando, nos solicita la contraseña del keystore, (introducimos por ejemplo **123456**), nombre, etc. Observa la captura:

```
usuario@usuario-MRC-WX0: ~$ keytool -genkey -alias claveSSL -keyalg RSA -keystore AlmacenSSL
Enter keystore password:
Re-enter new password:
Enter the distinguished name, Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces.
What is your first and last name?
[Unknown]: Ministerio de Educación, Formación Profesional y Deportes
What is the name of your organizational unit?
[Unknown]: CIDEAD
What is the name of your organization?
[Unknown]: Formación a Distancia
What is the name of your City or Locality?
[Unknown]: Sevilla
What is the name of your State or Province?
[Unknown]: Sevilla
What is the two-letter country code for this unit?
[Unknown]: ES
Is CN=Ministerio de Educación, Formación Profesional y Deportes, OU=CIDEAD, O=Formación a Distancia, L=Sevilla, ST=Sevilla, C=ES correct?
[no]: yes
Generating 3,072 bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 90 days
for: CN=Ministerio de Educación, Formación Profesional y Deportes, OU=CIDEAD, O=Formación a Distancia, L=Sevilla, ST=Sevilla, C=ES
usuario@usuario-MRC-WX0: ~$
```

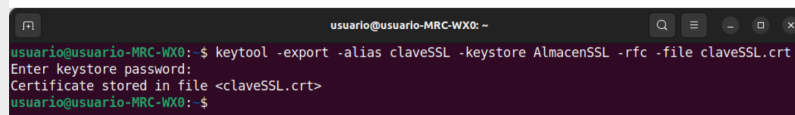
Exportar el certificado a un archivo (I)

El siguiente comando exporta el certificado de nombre `claveSSL`, almacenado en el keystore de nombre `AlmacenSSL` al fichero de nombre `claveSSL.crt`

```
keytool -export -alias claveSSL -keystore AlmacenSSL -rfc -file claveSSL.crt
```

Exportar el certificado a un archivo (II)

- Al introducir el comando nos solicita la contraseña del almacén de claves.
- Una vez introducida la contraseña, nos indica el resultado de la exportación.



```
usuario@usuario-MRC-WX0: ~  
usuario@usuario-MRC-WX0: $ keytool -export -alias claveSSL -keystore AlmacenSSL -rfc -file claveSSL.crt  
Enter keystore password:  
Certificate stored in file <claveSSL.crt>  
usuario@usuario-MRC-WX0: $
```

Importar el certificado al Truststore (I)

El siguiente comando importa el certificado `claveSSL.crt` bajo el nombre `claveTSSL` al almacén de nombre `TrustSSL`

```
keytool -import -alias clave_TSSL -file claveSSL.crt -keystore TrustSSL
```

Importar el archivo al Truststore (II)

- Al introducir el comando nos solicita la contraseña del almacén de claves.
- Una vez introducida la contraseña, nos indica el resultado de la importación.

```
usuario@usuario-MBC-WEB:~$ keytool -import -alias clave_TSSL -file claveSSL.crt -keystore TrustSSL
Enter keystore password:
Re-enter new password:
Owner: CN="Ministerio de Educación, Formación Profesional y Deportes", OU=CIDEAD, O=Formación a Distancia, L=Sevilla, ST=Sevilla, C=ES
Issuer: CN="Ministerio de Educación, Formación Profesional y Deportes", OU=CIDEAD, O=Formación a Distancia, L=Sevilla, ST=Sevilla, C=ES
Serial number: 332e68326c531d
Valid from: Thu May 15 18:17:41 CEST 2025 until: Wed Aug 13 18:17:41 CEST 2025
Certificate fingerprints:
    SHA1: 66:91:11:85:18:79:88:88:81:D1:4A:D3:13:84:24:69:FE:AE:80:1F
    SHA256: 4A:BA:D0:3D:35:6F:F1:3A:18:33:00:D0:1B:75:73:7D:AB:59:67:E5:EC:82:0F:23:33:98:ED:A4:96:5A:39:04
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 3072-bit RSA key
Version: 3
Extensions:
    #1: ObjectID: 2.5.29.14 Criticality=false
    SubjectKeyIdentifier [
        KeyIdentifier [
            0000: 77 88 FF 8F AE 1C 88 99 5D FC 82 7E 08 95 2F BC .....].....
            0010: 59 74 88 03
            Yt...
        ]
    ]
Trust this certificate? [no]: yes
Certificate was added to keystore
usuario@usuario-MBC-WEB:~$
```

Credenciales

Datos de licencia

Todas las capturas de pantalla de esta presentación tienen como datos de licencia:

Autoría: Alejandro Leal Cruz

Licencia: Uso educativo-no comercial.

Procedencia: Captura de pantalla de una consola de Ubuntu 22.04 al ejecutar el comando 'keytool' de Java propiedad de Oracle Corporation.

En el siguiente enlace tienes un ejemplo de una aplicación echo que incluye seguridad entre servidor y clientes. El servidor crea una conexión mediante un socket servidor seguro que atenderá conexiones de clientes que se identifiquen con un certificado válido. El certificado, tal y como hemos visto en la presentación anterior, se genera con la herramienta keytool. Además el ejemplo indica cómo ejecutar el proceso servidor y los procesos clientes:

[Ejemplo de aplicación que hace uso de sockets seguros en Java](#) 



Citas para pensar

"Dime y lo olvido, enséñame y lo recuerdo, involúcrame y lo aprendo".

Benjamin Franklin.



Autoevaluación

Señala la opción correcta. La herramienta keytool:

- ☐ Almacena certificados digitales.
- ☐ Permite validar certificados digitales.
- ☐ Es como un `keystore` pero solo para servidor.

☐ Permite generar certificados digitales.

No es correcto, prueba de nuevo.

No es correcto, deberías haber leído mejor.

No es correcto, debes prestar más atención.

Correcto. Muy bien, vamos por buen camino.

Solución

1. Incorrecto (#answer-78_63)
2. Incorrecto (#answer-78_81)
3. Incorrecto (#answer-78_84)
4. Opción correcta (#answer-78_87)



Para saber más

En el siguiente enlace encontrarás información detallada sobre el uso de la herramienta **Keytool** para manejar los certificados necesarios para la activación de SSL:

Manejo de certificados SSL con herramienta 'Keytool'  (<http://www.adictosaltrabajo.com/tutoriales/tutoriales.php?pagina=securitySSLKeytool>).